Table des matières

Exercices corrigés 11 — Contrôler la qualité 1
Questions courtes 1
Questions pratiques 2
Diagnostics 3
Questions à développer 4
Mises en situation 4
Corrigé 5

Exercices corrigés 11 — Contrôler la qualité

Comment travailler ces exercices. Répondez seul, puis vérifiez avec le corrigé. Les exercices marqués [labo] portent sur les contrats que vous venez d'écrire et sur les chiffres que vous avez sous les yeux ; les autres vérifient que le cours est clair dans votre tête. Tout vient de la Théorie 11 et du Labo 11.

Les chiffres à retrouver. Les questions pratiques supposent le labo fini : les quatre contrats en place, les deux rejeux de l'usine faits, le second avec les points douteux et les coupures en suivi. Les requêtes se tapent dans la ligne de commande DuckDB, lancée depuis `Projet_Boreal_intermediaire`, l'entrepôt ouvert en lecture seule par `ATTACH IF NOT EXISTS 'plateforme/entrepot/boreal.duckdb' AS entrepot (READ_ONLY); USE entrepot;` quand elles lisent la `RAW` ou les traces ; les commandes du pipeline, dans l'Anaconda Prompt, depuis `01_de`, la ligne de commande DuckDB fermée.

Le corrigé suit les énoncés, en fin de document. Répondez à l'énoncé en entier avant d'y passer : c'est en butant sur une question qu'on découvre ce qu'on avait lu sans le comprendre.

Questions courtes

1. Deux pannes. Dites pourquoi un pipeline qui échoue est une panne moins grave qu'un pipeline qui passe au vert sur une donnée fausse, et donnez un exemple de chacune chez Boréal.
2. Test ou contrat. Dites ce que vérifient les tests du Cours 05 et ce que vérifie un contrat, puis pourquoi aucun test ne pouvait voir le champ renommé du micrologiciel 2.0.
3. Ce que porte un contrat. Nommez les rubriques d'un contrat de données, et donnez leur valeur pour le contrat de la télémétrie.

© 2026 iAlexMG. Tous droits réservés.

4. Les deux côtés. Dans quels contrats du labo Data Engineering est-il le consommateur, et dans lequel est-il le producteur ? Nommez les consommateurs de ce dernier.
5. Neuf dimensions. Rangez chacune des six règles du contrat de l'ERP dans sa dimension.
6. La fraîcheur. Expliquez pourquoi une règle de fraîcheur ne peut pas se contenter de compter ce qui est là, et pourquoi celle de la télémétrie lit le journal des chargements plutôt que la table.
7. Trois sévérités. Pour une règle bloquante, d'avertissement et de suivi qui rompt, dites ce qui s'écrit au journal, ce qui s'écrit dans `pipeline.controles` et ce que devient le code de sortie.
8. Bloquer n'est pas refuser. Dites ce qu'une règle bloquante arrête chez Boréal, ce qu'elle n'arrête pas, et pourquoi.
9. Tout juger avant de décider. Pourquoi le module juge-t-il toutes les règles avant de lever `ContratRompu`, plutôt que de s'arrêter à la première rompue ?
10. La fenêtre dans les contrats. Dites ce que la fenêtre du Cours 06 change au contrat de l'ERP, et pour quel défaut.
11. Trois chiffres. Donnez un changement de contrat qui fait monter chacun des trois chiffres de la version, avec un exemple tiré de Boréal pour le premier.
12. Quatre classes. Définissez une alerte vraie, fausse, manquée, et vraie mais inutile, avec un exemple du rejeu de l'usine pour chacune.
13. Trois questions. Nommez les trois questions qui jugent une alerte avant qu'elle sonne, et dites ce qu'est une alerte qui n'y répond pas.
14. Les traces. Nommez les trois tables de traces de l'entrepôt, le cours qui a créé chacune, et une question à laquelle chacune répond.
15. Cinq temps. Nommez dans l'ordre les cinq temps de la conduite d'un incident de données, et dites ce qu'un post-mortem sans blâme cherche.

Questions pratiques

1. [labo] Le plus petit extrait. Écrivez la requête qui donne les cinq extraits de `VBAK` qui comptent le moins de commandes. Dites ce que ces mois ont en commun, et pourquoi le seuil de la règle de volume est à 300 plutôt qu'à la moyenne des extraits.
2. [labo] Le concessionnaire en retard. Retrouvez la commande qui rompt `concessionnaire_connu` : son numéro, son concessionnaire, sa date de création, et le premier lot où la fiche de ce concessionnaire arrive. Dites pourquoi la règle avertit au lieu de bloquer.
3. [labo] Par fichier ou par ligne. Comptez, table par table, les fichiers de l'ERP dont l'empreinte est celle d'un fichier d'un lot antérieur de la même table. Dites ce que donnerait une règle de relivraison écrite sur l'empreinte des fichiers.
4. [labo] Le capteur figé. Retrouvez l'étiquette, la journée, la valeur et le nombre de points du capteur figé. Relancez ensuite la même recherche sans la condition `MIN(p.Value) <> 0`, et expliquez ce qui sort.

© 2026 iAlexMG. Tous droits réservés.

5. [labo] Le nombre de points. Écrivez, dans la ligne de commande DuckDB, la règle de distribution sur le nombre de points de pression pendant la marche des lignes, au double de la médiane des vingt journées de production précédentes. Comptez ses alertes vraies et fausses, et comparez-la à la règle de l'écart-type.

6. [labo] Avril pour tous. Jugez les quatre contrats sur avril 2026, d'une seule commande. Dites quelles règles rompent, avec leurs valeurs, et ce que le contrat de l'argent ne dit pas.

7. [labo] Les dix heures. Retrouvez les dix heures de l'argent dont la moyenne sort de la plage du capteur : leurs étiquettes, le signe de leurs moyennes, et les points douteux qu'elles portent. Expliquez le lien avec un choix du Cours 08.

8. [labo] La semaine du 20 octobre. Écrivez la requête sur `pipeline.controles` qui donne, pour les journées du 20 au 26 octobre 2025 et le contrat de l'usine, le dernier verdict de chaque règle et de chaque journée, puis compte, règle par règle, les journées hors cible.

9. [labo] Les messages en retard. Comptez les messages de télémétrie reçus plus d'une heure, puis plus d'un jour, après la fin de leur sortie, et donnez le plus grand retard. Proposez une règle pour ce défaut, avec sa dimension et sa sévérité.

10. [labo] Les doublons permis. Comptez les messages de la télémétrie, leurs identifiants distincts et les messages en trop, puis prouvez qu'aucun identifiant ne porte deux contenus différents.

Diagnostics

1. En recopiant la règle `commande_relivree_a_l_identique`, un collègue oublie l'étoile devant `COLUMNS`. Le contrat passe, et la règle rend 27 746 commandes relivrées au lieu de 609. Expliquez.

2. Une développeuse retire les parenthèses de `z.SERNR = (t.message->>'sernr')`, qu'elle juge décoratives. Le contrat de la télémétrie s'arrête sur `Conversion Error: Malformed JSON at byte 0 of input: unexpected character. Input: "HR23000560" when casting from source column SERNR`. Expliquez le message, valeur comprise.

3. Le 1er mai 2025, pour que le pipeline passe, un ingénieur fait de `champs_du_message` une règle d'avertissement dans le contrat 1.0.0. Le pipeline repasse au vert. Dites ce qui est réglé, ce qui ne l'est pas, et ce qu'il fallait faire.

4. Un analyste retire du capteur figé la condition `MIN(p.Value) <> 0`, pour « ne rien laisser passer ». Rejouée sur 182 jours, la règle trouve 359 journées sur 14 étiquettes, dont 108 pour `UTL.CMP3.PUISSANCE`. Expliquez.

5. Pour simplifier, une collègue écrit la fraîcheur de la télémétrie sur la table plutôt que sur le journal : `table: raw.telemetrie_sorties`, `periode: _lot`. Jugée sur août et septembre 2023, la règle trouve 41 jours manquants ; celle du labo, 0. Expliquez, et dites laquelle a raison.

6. En réécrivant la règle de pression, un ingénieur pose `{fenetre}` dans la construction des journées, sous `WHERE lignes > 0`, plutôt que sur le jugement. Jugée sur le 2025-10-20, la règle rend 0 au lieu de 1. Expliquez.

© 2026 iAlexMG. Tous droits réservés.

7. Pour montrer le blocage d'avril, un formateur déclenche le DAG mensuel pour le 2026-04-01. L'exécution passe à `success` en quelques millisecondes, et le formateur conclut que le contrat ne sert à rien. Expliquez ce qui s'est passé, et comment juger avril par Airflow.

8. Pour rattraper les 15, 16 et 17 octobre, que la règle de pression manque, un ingénieur abaisse son seuil du double de la médiane à une fois et demie. Le rejeu trouve alors une fausse alerte, le 4 décembre, et manque toujours les trois journées ; à une fois et quart, neuf fausses. Dites pourquoi baisser le seuil ne rattrape rien, et ce qu'il fallait faire.

Questions à développer

A. « Les tests dbt d'Analytics Engineering suffisent : ils vérifient déjà l'unicité, les valeurs acceptées et les relations. » Discutez, en vous appuyant sur trois défauts de Boréal que ces tests ne pourraient pas voir, puis dites ce que chacun des deux fait mieux que l'autre.

B. « Il serait plus sûr de refuser la donnée qui rompt un contrat, dès l'entrée. » Discutez, avec l'extrait livré deux fois, le micrologiciel 2.0 et la pression instable, et dites ce que la `RAW` perdrait à devenir un filtre.

C. Le rejeu de l'usine laisse huit alertes en six mois, et deux règles en suivi. Proposez qui reçoit chaque alerte des quatre contrats, par quel moyen, et à quel rythme, puis dites comment vous saurez, dans un an, si ces alertes sont encore lues.

Mises en situation

1. Une fromagerie fictive de Victoriaville reçoit chaque nuit un fichier CSV par producteur de lait, une soixantaine en tout, avec le volume livré et les résultats du laboratoire d'analyse, dont la matière grasse. Certains soirs, un producteur n'envoie rien ; parfois, le même fichier arrive deux fois ; au printemps, le laboratoire annonce qu'il passera la matière grasse d'un pourcentage à des grammes par litre. Écrivez les règles de son contrat, avec leur dimension et leur sévérité, et dites comment la version du contrat suivra le changement du laboratoire.

2. Un réseau fictif de bornes de recharge de Rivière-du-Loup reçoit de chaque borne un message par recharge. Une mise à jour du fabricant fait passer le champ `energie` des kilowattheures aux wattheures, sans changer son nom. Dites quelles règles d'un contrat verraient ce changement, et lesquelles non, puis ce que le contrat aurait dû exiger du fabricant.

3. Une mine fictive de Chibougamau reçoit environ 300 alertes par jour de ses capteurs de ventilation. Les opérateurs ont cessé de les lire, et une vraie montée de monoxyde de carbone est passée inaperçue pendant une heure. Proposez une démarche, étape par étape, pour ramener ces alertes à ce qu'une personne peut lire, sans perdre celles qui comptent.

4. (avancé) Chez un distributeur alimentaire fictif de Drummondville, l'extrait des factures donne, pendant trois semaines, les prix en cents au lieu de dollars, et le tableau de bord du chiffre d'affaires est multiplié par cent sans que personne ne s'en rende compte avant la clôture du mois. Écrivez le plan du post-mortem : le déroulé, l'impact, la cause, les actions. Dites quelles règles d'un contrat auraient arrêté l'incident, et à quel moment.

© 2026 iAlexMG. Tous droits réservés.

Corrigé

Les chiffres cités sont ceux de la génération du cours, jusqu'à 2026-03, et du labo rejoué dans l'ordre du lecteur. Les identifiants des passages, les durées et les dates des exécutions d'Airflow changent d'un passage à l'autre ; les comptes, eux, ne changent pas.

Questions courtes

1. Deux pannes. Un pipeline qui échoue le dit : un code de sortie, une ligne « échoué » dans `pipeline.executions`, une tâche rouge. Quelqu'un le voit, et rien de faux ne part en aval. Un pipeline qui passe au vert sur une donnée fausse ne dit rien, et la donnée fausse voyage jusqu'à une décision. Chez Boréal, l'extraction ODBC sans pilote échoue franchement (Cours 04) ; le mois d'avril 2026 demandé à la `RAW` passe au vert, zéro ligne et code 0.

Théorie : `Theorie_11_DE.md`, section 1.

2. Test ou contrat. Les tests du Cours 05 vérifient le code sur des données qu'ils fabriquent : que le pipeline fait ce qu'on attend de lui. Un contrat vérifie la donnée qui arrive, à chaque livraison. Le code du pipeline n'a pas changé le 1er mai 2025 : c'est le message qui a changé, et il est entré tel quel, en JSON. Aucun test ne pouvait le voir, parce que le défaut n'était pas dans le code.

Théorie : `Theorie_11_DE.md`, section 1.

3. Ce que porte un contrat. Le produit, le producteur, les consommateurs, la version, la fenêtre et les règles. Pour la télémétrie : le produit est `raw.telemetrie_sorties` ; le producteur, le module télématique des véhicules, par le courtier MQTT et l'abonné de Boréal ; les consommateurs, Analytics Engineering et Data Science ; la version, 2.0.0 ; la fenêtre, la colonne `_lot` ; et quatre règles, la fraîcheur, les champs du message, l'identifiant sans conflit et le numéro de série connu.

Théorie : `Theorie_11_DE.md`, section 2.

4. Les deux côtés. Data Engineering est le consommateur des contrats de l'ERP, de la télémétrie et de l'historien : il reçoit ces livraisons. Il est le producteur du contrat de l'argent, les séries horaires de l'usine qu'il livre ; ses consommateurs sont Analytics Engineering, Data Science et Business Intelligence.

Théorie : `Theorie_11_DE.md`, section 2.

5. Neuf dimensions. `extrait_du_mois_livre`, la fraîcheur ; `commandes_du_mois`, le volume ; `commande_unique_dans_son_lot` et `commande_relivree_a_l_identique`, l'unicité ; `numeros_au_format_sap`, la validité ; `concessionnaire_connu`, l'intégrité.

Théorie : `Theorie_11_DE.md`, section 3.

6. La fraîcheur. Une table ne peut pas montrer une ligne qu'elle n'a jamais reçue : compter ce qui est là ne dit rien de ce qui manque. La règle doit comparer ce qui est là à ce qui était attendu, la liste des jours ou des mois de la fenêtre. Celle de la télémétrie lit le journal des chargements, parce que 41 archives de 2023 sont vides : livrées, elles ne portent aucun message, et la table ne sait rien d'elles.

Théorie : `Theorie_11_DE.md`, section 3.

© 2026 iAlexMG. Tous droits réservés.

7. Trois sévérités. Rompue, une règle bloquante s'écrit au niveau `ERROR`, « ROMPUE », et fait lever `ContratRompu` à la fin : code 1. Une règle d'avertissement s'écrit au niveau `WARNING`, « à voir », et laisse le code à 0. Une règle de suivi s'écrit au niveau `INFO`, « hors cible », et laisse le code à 0. Les trois s'inscrivent de la même façon dans `pipeline.controles`, avec `respectee` à faux.

Théorie : `Theorie_11_DE.md`, section 4.

8. Bloquer n'est pas refuser. Une règle bloquante arrête ce qui se construirait sur la donnée : la tâche des contrats échoue, et `dbt_build` ne part pas. Elle n'arrête pas la donnée : les contrats tournent après le chargement de la `RAW`, qui garde tout ce qui est arrivé. Une donnée refusée serait perdue, et la `RAW` cesserait d'être le brut ; jugée et gardée, elle se relit une fois le défaut compris.

Théorie : `Theorie_11_DE.md`, section 4.

9. Tout juger avant de décider. Une exécution qui s'arrêterait à la première règle rompue cacherait les suivantes : on corrigerait un défaut, on relancerait, on découvrirait le suivant. En jugeant tout, le passage d'avril 2026 montre d'un coup ses deux règles bloquantes, et la table garde le verdict des quatre autres.

Théorie : `Theorie_11_DE.md`, section 4.

10. La fenêtre dans les contrats. Sans fenêtre, le contrat juge tout l'historique et sonne à chaque passage pour l'extrait de mars 2023 livré deux fois, connu depuis trois ans. Avec la fenêtre du mois, il ne sonne qu'une fois, le mois où l'extrait arrive, et le lot `2023-03_bis` garde son mois. Au labo, le rejeu mois par mois de l'ERP donne deux alertes en 51 mois.

Théorie : `Theorie_11_DE.md`, section 4.

11. Trois chiffres. Le premier monte quand un changement casse la lecture : un champ renommé, supprimé, ou dont le type change ; chez Boréal, la télémétrie passée de 1.0.0 à 2.0.0 quand le micrologiciel 2.0 a renommé `temp_moteur_max`. Le deuxième monte quand on ajoute sans rien casser : une valeur de plus dans une liste, un champ facultatif. Le troisième, pour une correction qui ne change rien au consommateur : une description, une règle mieux écrite.

Théorie : `Theorie_11_DE.md`, section 5.

12. Quatre classes. Vraie : elle tombe pendant un événement réel, la pression instable du 20 octobre. Fausse : elle tombe hors de tout événement ; aucune après la correction de la règle de pression, huit avant, sur des journées courtes. Manquée : un jour d'événement sans alerte, les 15, 16 et 17 octobre. Vraie mais inutile : l'événement est réel, mais personne n'a rien à faire ce jour-là ; les points douteux, 55 journées, et les coupures, 44.

Théorie : `Theorie_11_DE.md`, section 7.

13. Trois questions. Qui la reçoit ? Que fera cette personne ? Combien de fois par mois sonnera-t-elle ? Une alerte sans destinataire, sans action ou qui sonne tous les jours n'est pas une alerte : c'est un journal.

Théorie : `Theorie_11_DE.md`, section 7.

© 2026 iAlexMG. Tous droits réservés.

14. Les traces. `pipeline.chargements`, du Cours 03 : qu'est-ce qui est arrivé, quand, en quelle quantité, avec quelle empreinte ? `pipeline.executions`, du Cours 05 : quelle étape a tourné, combien de temps, et a-t-elle fini ? `pipeline.controles`, de ce cours : que valait la donnée quand elle est passée, règle par règle et fenêtre par fenêtre ?

Théorie : `Theorie_11_DE.md`, section 8.

15. Cinq temps. Détecter, qualifier, contenir, corriger, apprendre. Le post-mortem sans blâme cherche ce qui, dans le système, a laissé passer l'erreur, et non qui l'a commise : il finit par des actions qui changent le système, chacune avec un responsable et une date.

Théorie : `Theorie_11_DE.md`, section 9.

Questions pratiques

1. [labo] Le plus petit extrait.

```sql
SELECT _lot, COUNT(*) AS commandes FROM raw.erp_vbak
GROUP BY ALL ORDER BY commandes LIMIT 5;
```

_lot varchar	commandes int64
2024-12	380
2023-12	382
2025-12	427
2022-12	429
2022-08	448

Quatre mois de décembre et un mois d'août : des mois creux, qui reviennent chaque année. La moyenne des extraits ferait sonner la règle chaque décembre, pour un mois tranquille et complet. Le seuil de 300, sous le plus petit mois jamais livré, ne sonne que pour un extrait tronqué.

Théorie : `Theorie_11_DE.md`, section 3.

2. [labo] Le concessionnaire en retard.

```sql
SELECT v._lot, v.VBELN, v.KUNNR, v.ERDAT
FROM raw.erp_vbak v
WHERE NOT EXISTS (SELECT 1 FROM raw.erp_kna1 k WHERE k.KUNNR = v.KUNNR AND k._lot <= v._lot);

SELECT _lot, KUNNR, NAME1, ERDAT FROM raw.erp_kna1
WHERE KUNNR = '0000200154' ORDER BY _lot LIMIT 1;
```


© 2026 iAlexMG. Tous droits réservés.

La commande `0010015405`, du lot `2024-05`, créée le 31 mai 2024 (`20240531`), pour le concessionnaire `0000200154`, Yarrowby-Dunn Recreation. Sa fiche arrive au lot `2024-06`, créée le même 31 mai : le concessionnaire, né le dernier jour du mois, n'entre dans les données maîtres qu'à l'extrait suivant. La règle avertit parce que le défaut se résout seul le mois suivant, et que le `STAGING` d'Analytics Engineering retrouve la fiche au lot d'après.

Théorie : `Theorie_11_DE.md`, section 4.

3. [labo] Par fichier ou par ligne.

```sql
SELECT table_raw, COUNT(*) AS fichiers_deja_vus
FROM pipeline.chargements a
WHERE lignes > 0 AND lot <> 'complet'
  AND EXISTS (SELECT 1 FROM pipeline.chargements b
              WHERE b.empreinte = a.empreinte AND b.table_raw = a.table_raw AND b.lot < a.lot)
GROUP BY ALL ORDER BY ALL;
```

table_raw varchar	fichiers_deja_vus int64
erp_kna1	40
erp_makt	45
erp_mara	42
erp_t005u	51
erp_vbak	1
erp_vbap	1
erp_zserie	1

Une règle par fichier sonnerait 181 fois. Seules les trois dernières lignes parlent de commandes, de postes et de numéros de série livrés deux fois, ceux du lot `2023-03_bis`. Les 178 autres sont des données maîtres, concessionnaires, articles, libellés et régions, qui reviennent identiques d'un extrait à l'autre quand rien n'a changé : c'est leur état normal. Comparer la ligne entière, lignage exclu, ne retient que les 609 commandes du lot relivré.

Théorie : `Theorie_11_DE.md`, section 8.

4. [labo] Le capteur figé. Dans la ligne de commande DuckDB, sans `ATTACH` :

```sql
CREATE OR REPLACE VIEW points AS
SELECT * FROM read_parquet('plateforme/lac/bronze/usine/historien/**/*.parquet',
                           hive_partitioning = true)
WHERE CAST(Timestamp AS DATE) = jour;
CREATE OR REPLACE TABLE etiquettes AS
SELECT * FROM read_csv('sources/usine/referentiel/etiquettes.csv');
CREATE OR REPLACE TABLE jours_de_production AS
```


© 2026 iAlexMG. Tous droits réservés.

```sql
SELECT jour FROM points WHERE TagName LIKE '%.CNV.COMPTEUR' AND Quality = 192
GROUP BY jour HAVING MAX(Value) > 0;

SELECT p.TagName, p.jour, MIN(p.Value) AS valeur, COUNT(*) AS points
FROM points p JOIN etiquettes e ON e.etiquette = p.TagName
WHERE e.bande_morte > 0 AND p.Quality = 192 AND p.jour IN (FROM jours_de_production)
GROUP BY ALL HAVING COUNT(DISTINCT p.Value) = 1 AND MIN(p.Value) <> 0;
```

`PEI.CAB.HUMIDITE`, l'humidité de la cabine de peinture, le 2025-10-08 : 24 points, tous à 54,52 %. Une humidité qui ne bouge pas d'un centième en une journée de production, avec une bande morte de 0,8, c'est un capteur qui ne mesure plus. Sans la condition sur le zéro, la recherche rend 359 journées sur 14 étiquettes : l'humidité figée, et 358 journées d'équipements à l'arrêt, dont la puissance ou la vitesse vaut zéro toute la journée, le compresseur 3, d'appoint, en tête. Le zéro d'un équipement arrêté est une vraie mesure, pas un capteur figé.

Théorie : `Theorie_11_DE.md`, section 3.

5. [labo] Le nombre de points. Avec la table `journees` de l'étape 7 du labo, dans sa seconde version, qui porte la colonne `points` :

```sql
SELECT jour, points, reference
FROM (SELECT jour, points, MEDIAN(points) OVER vingt AS reference, COUNT(*) OVER vingt AS jours
      FROM journees
      WINDOW vingt AS (ORDER BY jour ROWS BETWEEN 20 PRECEDING AND 1 PRECEDING))
WHERE jours >= 5 AND points > 2 * reference
ORDER BY jour;
```

jour date	points int64	reference double
2025-10-17	28	12.0
2025-10-20	118	12.5
2025-10-21	161	13.0
2025-10-22	148	13.0
2025-10-23	175	13.0
2025-10-24	174	13.0
2025-10-27	157	13.0
2025-10-28	91	14.0

Huit alertes, toutes pendant la dérive, zéro fausse, et le 17 octobre en plus : un jour ouvrable plus tôt que la règle de l'écart-type. La bande morte de l'historien écrit un point chaque fois que la valeur bouge assez ; une pression qui commence à osciller écrit plus de points avant que son écart-type décolle. Les deux règles se complètent : le volume de points voit le début, l'écart-type mesure l'ampleur.

Théorie : `Theorie_11_DE.md`, section 6.

© 2026 iAlexMG. Tous droits réservés.

6. [labo] Avril pour tous.

   ```
   python -m pipeline contrats --depuis 2026-04 --jusqua 2026-04
   ```

   Quatre règles bloquantes rompent, code 1 : `extrait_du_mois_livre`, 1 mois manquant ; `commandes_du_mois`, 0 commande ; `archive_du_jour_livree`, 30 jours ; `export_du_jour_livre`, 30 jours. Les 18 autres sont respectées. Le contrat de l'argent ne dit rien : ses cinq règles sont respectées, parce qu'aucune ne demande que les heures d'avril soient là. Il lui manque une règle de fraîcheur, par exemple `periodes_manquantes` sur `argent.usine_horaire`, par jour.

Théorie : `Theorie_11_DE.md`, section 3.

7. [labo] Les dix heures.

   ```
   SELECT u.etiquette, u.heure, ROUND(u.valeur, 1) AS moyenne, u.points, u.points_douteux,
          e.plage_min, e.plage_max
   FROM argent.usine_horaire u JOIN raw.usine_etiquettes e ON e.etiquette = u.etiquette
   WHERE NOT u.valeur BETWEEN CAST(e.plage_min AS DOUBLE) AND CAST(e.plage_max AS DOUBLE)
   ORDER BY u.heure;
   ```

   Dix heures, dix étiquettes différentes, toutes avec une moyenne négative, de −0,1 pour la vibration d'un robot à −88,8 pour la pression d'air du poste de vissage des motoneiges, sous une plage qui commence à 0. Chacune porte exactement un point douteux, et les dix n'en comptent que 37 en tout. Le Cours 08 a choisi de compter une valeur douteuse plutôt que de l'écarter, et une seule valeur aberrante suffit à tirer hors de la plage la moyenne d'une heure qui compte peu de points. Le contrat le rend visible sans corriger le choix.

Théorie : `Theorie_11_DE.md`, section 2.

8. [labo] La semaine du 20 octobre.

   ```
   SELECT regle, severite, COUNT(*) AS journees, SUM(valeur) AS total
   FROM (SELECT * FROM pipeline.controles
         WHERE contrat = 'usine_historien'
           AND fenetre BETWEEN 'du 2025-10-20 au 2025-10-20' AND 'du 2025-10-26 au 2025-10-26'
         QUALIFY ROW_NUMBER() OVER (PARTITION BY fenetre, regle ORDER BY controle_le DESC) = 1)
   WHERE NOT respectee
   GROUP BY ALL ORDER BY ALL;
   ```

regle varchar	severite varchar	journees int64	total double
coupures_de_communication	suivi	2	16.0
points_douteux	suivi	1	1.0

© 2026 iAlexMG. Tous droits réservés.

```
┌──────────────────────┬──────────────┬──────────┬──────┐
│ pression_instable    │ avertissement│        5 │  5.0 │
└──────────────────────┴──────────────┴──────────┴──────┘
```

Chaque journée a été jugée deux fois, par les deux rejeux : `QUALIFY` ne garde que le verdict le plus récent, celui où les deux premières règles sont en suivi. La comparaison sur le texte de la fenêtre marche parce qu'elle commence par la date. Cinq journées de pression instable, du lundi au vendredi, les deux jours de fin de semaine n'étant pas des journées de production.

Théorie : `Theorie_11_DE.md`, section 8.

9. [labo] Les messages en retard.

```sql
SELECT COUNT(*) FILTER (WHERE recu - fin > INTERVAL 1 HOUR) AS plus_d_une_heure,
       COUNT(*) FILTER (WHERE recu - fin > INTERVAL 1 DAY) AS plus_d_un_jour,
       MAX(recu - fin) AS retard_max
FROM (SELECT CAST(recu_le AS TIMESTAMPTZ) AS recu,
             CAST(message->>'fin' AS TIMESTAMPTZ) AS fin
      FROM raw.telemetrie_sorties);
```

8 469 messages reçus plus d'une heure après la fin de leur sortie, 430 plus d'un jour après, et au plus 4 jours 1 heure 17 secondes. Un véhicule sans réseau garde ses résumés et les envoie au retour. Une règle de fraîcheur des messages, par exemple « la part des messages reçus plus d'un jour après leur fin reste sous 1 % », se range en avertissement ou en suivi : le retard est normal pour un véhicule en forêt, et bloquer le pipeline pour lui ne servirait à rien. Elle servirait à voir un courtier ou un abonné qui prend du retard.

Théorie : `Theorie_11_DE.md`, sections 3 et 4.

10. [labo] Les doublons permis.

```sql
SELECT COUNT(*) AS messages, COUNT(DISTINCT message->>'message_id') AS identifiants,
       COUNT(*) - COUNT(DISTINCT message->>'message_id') AS en_trop
FROM raw.telemetrie_sorties;

SELECT COUNT(*) AS en_conflit FROM (
  SELECT message->>'message_id' FROM raw.telemetrie_sorties
  GROUP BY 1 HAVING COUNT(DISTINCT CAST(message AS VARCHAR)) > 1);
```

204 485 messages, 201 542 identifiants, 2 943 messages en trop, et 0 identifiant en conflit : chaque doublon est une copie exacte, le prix d'une livraison « au moins une fois » (Cours 09). C'est ce que dit la règle `identifiant_sans_conflit`, qui permet la copie et refuse la contradiction.

Théorie : `Theorie_11_DE.md`, section 3.

Diagnostics

© 2026 iAlexMG. Tous droits réservés.

1. Sans l'étoile, DuckDB déplie `COLUMNS(...)` en autant d'expressions qu'il y a de colonnes, une empreinte par colonne, toutes nommées `empreinte`. La suite de la requête n'en lit qu'une, la première, celle de `MANDT`, le mandant SAP, qui vaut la même chose sur toutes les lignes. Toute ligne a alors la même empreinte qu'une ligne d'un lot antérieur, sauf celles du premier extrait : 28 271 lignes moins les 525 de janvier 2022 font 27 746. `*COLUMNS(...)` passe les colonnes comme arguments d'un seul appel, et l'empreinte couvre la ligne entière.

Labo : `Labo_11_DE.md`, étape 3.

2. `->>` se lie moins fort que `=` : DuckDB lit `(z.SERNR = t.message)->>'sernr'`. Pour comparer `z.SERNR`, du texte, à `t.message`, du JSON, il convertit le numéro de série en JSON, et `HR23000560`, un numéro de série de `ZSERIE` tel que l'ERP le livre, n'est pas du JSON valide sans guillemets : d'où « Malformed JSON at byte 0 ». Les parenthèses forcent l'extraction avant la comparaison.

Labo : `Labo_11_DE.md`, étape 4.

3. Le pipeline passe, et c'est tout ce qui est réglé. Le contrat 1.0.0 dit toujours que le message a treize champs, et il avertira chaque jour, de plus en plus fort, jusqu'à 115 807 messages hors contrat ; au bout d'une semaine, plus personne ne lira l'avertissement. Et rien n'est négocié : les consommateurs ne savent pas que la température a changé de nom, et leurs lecteurs de `temp_moteur_max` deviennent muets en août. Il fallait renégocier le contrat, avec le producteur et les consommateurs, et le passer en 2.0.0, qui déclare les deux formes, en faisant réécrire les lecteurs.

Théorie : `Theorie_11_DE.md`, section 5.

4. Une mesure d'un équipement à l'arrêt vaut zéro, fidèlement, toute la journée. Le compresseur 3, d'appoint, tourne rarement : sa puissance vaut zéro 108 journées de production sur 121. La règle sans la condition confond un capteur qui ne mesure plus avec un équipement qui ne tourne pas, et noie la seule vraie alerte, l'humidité du 8 octobre, dans 358 fausses. Le zéro s'écarte parce qu'il est la valeur normale d'un arrêt ; une valeur non nulle qui ne bouge pas, elle, n'a pas d'explication normale.

Théorie : `Theorie_11_DE.md`, sections 3 et 6.

5. La table ne connaît que les jours qui portent au moins un message. Les 41 archives vides de 2023, les 39 premières, du 1er août au 8 septembre, puis celles des 11 et 12 septembre, ont été livrées : aucune ligne de la `RAW` ne porte leur lot, et la règle écrite sur la table les compte comme manquantes. Le journal des chargements, lui, les a inscrites, avec zéro ligne. La règle du labo a raison : une archive vide est une archive livrée, et la fraîcheur juge la livraison, pas son contenu.

Théorie : `Theorie_11_DE.md`, section 3.

6. La référence d'une journée est la médiane des vingt journées de production qui la précèdent. Posée dans la construction, la fenêtre d'une journée ne laisse qu'une ligne dans `journees` : plus aucune journée avant elle, zéro jour de référence, une médiane vide, et la condition `jours >= 5` n'est jamais vraie. La règle ne peut plus alerter sur une fenêtre d'un jour, celle du DAG quotidien. La fenêtre se pose sur le jugement, pas sur ce dont le jugement a besoin.

Labo : `Labo_11_DE.md`, étape 7.

© 2026 iAlexMG. Tous droits réservés.

7. La date est après la borne de fin du DAG, le 2026-03-31. Airflow 3.3.1 crée l'exécution, n'y lance aucune tâche, et la marque réussie : `airflow tasks states-for-dag-run` répond `No data found`. Le contrat n'a jamais tourné. C'est exactement la panne que le cours chasse, un vert sans travail. Pour juger avril par Airflow, `airflow tasks test boreal_mensuel contrats_du_mois 2026-04-01` lance la tâche seule, et elle échoue sur deux règles bloquantes.

Labo : `Labo_11_DE.md`, étape 10.

8. Les 15, 16 et 17 octobre ont un écart-type de 4,2, 3,4 et 4,6, sous la médiane de leurs vingt journées précédentes ou tout près : aucun seuil au-dessus de la médiane ne les rattrape, et chaque seuil plus bas ajoute des journées ordinaires un peu agitées. L'écart-type n'a pas encore bougé ; c'est la mesure qui est aveugle, pas le seuil qui est trop haut. Il fallait une autre mesure, le nombre de points, qui double dès le 17 octobre, comme la pratique 5 le montre.

Théorie : `Theorie_11_DE.md`, section 6.

Questions à développer

A. Les tests dbt vérifient ce qu'on fait de la donnée, une fois qu'elle est dans le `STAGING`. Trois défauts de Boréal leur échappent. Le mois d'avril 2026 : un test d'unicité ou de relation sur une table vide passe, et seule une règle qui connaît ce qui était attendu voit le mois absent. Le champ renommé du micrologiciel 2.0 : un `STAGING` qui lit `temp_moteur_max` rend `NULL`, et un test `not_null` n'y serait que si quelqu'un avait pensé à le poser, sans dire pourquoi. La pression instable : c'est une vraie mesure, qu'aucun test de structure ne juge. Le contrat voit ce qui arrive, au plus tôt, là où l'on sait encore d'où vient la donnée, et bloque avant que quoi que ce soit se construise. Le test dbt voit ce qu'un modèle fait de travers : une jointure qui double des lignes, un filtre qui en perd, un calcul faux, que le contrat, en amont, ne peut pas voir. Les deux ont la même forme, une requête, un attendu, une sévérité ; ils se placent à deux endroits de la chaîne, et aucun ne remplace l'autre.

Théorie : `Theorie_11_DE.md`, sections 1 et 10.

B. Refuser à l'entrée semble plus sûr, et perd plus qu'il ne protège. L'extrait livré deux fois : refusé, il ne laisse aucune trace, et personne ne saura qu'il est arrivé ; gardé et marqué de son lot, il se voit, se compte, et le `STAGING` le traite. Le micrologiciel 2.0 : refuser les messages hors contrat du 1er mai, c'est perdre la télémétrie de toute la flotte à mesure qu'elle bascule, alors qu'il suffisait de renégocier le contrat et de relire les messages gardés. La pression instable : c'est une vraie mesure d'une vraie dérive, la refuser serait détruire la preuve de l'incident. Une `RAW` qui filtre cesse d'être le brut, et plus rien ne se rejoue à partir d'elle. Le bon geste est de juger dès l'entrée et d'arrêter ce qui se construit dessus, pas de jeter.

Théorie : `Theorie_11_DE.md`, section 4.

C. Les quatre contrats n'ont pas les mêmes destinataires. Les règles bloquantes de l'ERP, de la télémétrie et de l'historien vont à l'ingénieur de garde de Data Engineering, par le canal de garde, au moment où la tâche échoue, parce qu'il doit agir le jour même. Les avertissements de l'ERP, la commande relivrée et le concessionnaire en retard, vont à l'équipe de l'ERP et à Analytics Engineering, par courriel, groupés par jour. La pression instable et le capteur figé vont à la maintenance de l'usine, dans son propre outil, parce que c'est elle qui peut agir ; l'ingénieur de données n'y peut rien. Les deux règles de suivi alimentent un tableau de bord hebdomadaire, lu en réunion. Le contrat de l'argent avertit Data Engineering lui-même, producteur. Pour savoir dans un an si les alertes sont lues, on les compte, comme au rejeu : combien par mois pour chaque destinataire, combien ont mené à une action consignée, combien de journées d'événement sans alerte. Une règle qui sonne sans jamais mener à une action se passe en suivi ou se réécrit.

© 2026 iAlexMG. Tous droits réservés.

Théorie : `Theorie_11_DE.md`, section 7.

Mises en situation

1. Fromagerie de Victoriaville. Le contrat porte sur le fichier de chaque producteur, avec la livraison du soir comme fenêtre. Fraîcheur : un fichier par producteur attendu, comparé à la liste des producteurs actifs, en avertissement, parce qu'un producteur qui n'envoie rien un soir n'empêche pas de payer les autres. Unicité : un fichier relivré à l'identique, en avertissement, gardé et marqué. Validité : un volume positif, une matière grasse dans une plage plausible, en bloquant, parce qu'elle entre dans le prix payé au producteur. Schéma : les colonnes et leurs unités, en bloquant. Distribution : le volume d'un producteur comparé à ses vingt dernières livraisons, en avertissement. Le passage aux grammes par litre casse la lecture : le contrat passe en 2.0.0, négocié avant la date annoncée par le laboratoire, avec une colonne d'unité ou un nom neuf, et la plage de validité change avec lui.

Théorie : `Theorie_11_DE.md`, sections 3 et 5.

2. Bornes de Rivière-du-Loup. La règle de schéma ne voit rien : le champ garde son nom et son type. Une règle de validité sur la plage de l'énergie d'une recharge, par exemple de 0 à 150 kWh, la voit dès le premier message, puisque les valeurs sont mille fois plus grandes. Une règle de distribution, l'énergie moyenne d'une recharge comparée aux vingt jours précédents, la voit aussi. Le contrat aurait dû déclarer l'unité de chaque champ et exiger qu'un changement d'unité soit traité comme un changement qui casse : annoncé, négocié, et accompagné d'une nouvelle version du contrat, idéalement d'un nouveau nom de champ.

Théorie : `Theorie_11_DE.md`, sections 3 et 5.

3. Mine de Chibougamau. D'abord compter : rejouer un mois d'alertes et classer chacune, vraie, fausse, manquée ou vraie mais inutile, comme au labo. Puis regarder chaque fausse et la comprendre avant de corriger : un seuil fixe sur un capteur qui dérive, une galerie fermée dont le ventilateur est arrêté, un capteur en entretien. Corriger les règles sur la vraie sortie, en ajoutant l'état des équipements comme la marche des lignes au labo. Passer en suivi ce qui est réel mais n'appelle aucune action immédiate. Donner à chaque alerte un destinataire et une action écrite, et séparer ce qui touche la vie des mineurs, le monoxyde de carbone, qui doit sonner sans aucun filtre et être rejoué pour prouver qu'on n'en manque aucune. Enfin, compter chaque mois combien d'alertes sont envoyées et combien sont suivies d'une action.

Théorie : `Theorie_11_DE.md`, sections 6 et 7.

4. (avancé) Distributeur de Drummondville. Le déroulé : le jour du premier extrait en cents, les jours de facturation qui suivent, la clôture où l'écart est vu, la correction et le rechargement. L'impact : le chiffre d'affaires publié pendant trois semaines, les décisions prises dessus, les rapports envoyés, les commissions calculées. La cause, sans blâme : rien n'annonçait le changement d'unité, et rien ne vérifiait l'ordre de grandeur des prix. Les actions : une règle de validité, un prix unitaire dans une plage par famille de produits, en bloquant ; une règle de distribution, le montant total d'une journée comparé aux vingt précédentes, en avertissement ; l'unité déclarée au contrat, et tout changement annoncé ; la correction rejouée par la fenêtre des jours touchés, sans rien doubler. La règle de validité aurait arrêté l'incident le premier soir, avant que le tableau de bord se construise sur la donnée fausse.

Théorie : `Theorie_11_DE.md`, section 9.

© 2026 iAlexMG. Tous droits réservés.